

PY101: Week 1 - Introduction to Python

Lesson 2: Variables, Types, and Strings

Variables act as named storage locations for data in your code. Python handles structural text via strings, numbers via integers or floats, and truth values via booleans.

Why is 'NameError' so common?

A NameError usually happens because of a typo in the variable name, or because you are trying to use a variable before assigning it a value. Always double-check your spelling and ensure the variable is defined above the line where it is first used.

Lesson 6: Division Modes

Python offers multiple ways to divide numbers, processing standard true division with the `/` operator, floor division via `//`, and tracking remainders using the modulo operator.

When exactly should I use Modulo (%)?

Modulo is your go-to operator for any problem involving cycles or patterns. It's perfect for determining if a number is even or odd (`n % 2`), cycling through game turns, or finding out if a year is a leap year.

Lesson 8: Rounding and Money Format

Formatting output accurately is essential when presenting calculated values, ensuring that text values cleanly display numeric values, floats, and formatted financial statistics.

round() vs. f-string formatting

Use `round()` if you need to perform more math with the result, because it changes the value itself. Use `:.2f` inside an f-string only if you just want the output to *look* like currency for the user, while keeping the full-precision number for other calculations.

Lesson 11 & 12: Functions (Return vs. Print)

Functions isolate blocks of code. Understanding the explicit operational difference between showing a text evaluation in the console and returning a data value to the main program flow is key.

Is 'return' mandatory?

Technically, no. If a function doesn't have a `return` statement, Python automatically returns `None`. However, if you want your function to provide a result that your program can store or use later, you *must* use `return`.

Lesson 13: Scope and Local Variables

Variables created inside functions are restricted to that local function block, separating data namespaces from the broader global scope context.

Why avoid global variables?

Global variables can be changed from anywhere, making it very hard to debug when something goes wrong. Local variables (inside functions) are "contained"—the function is like a self-contained machine that you feed inputs (parameters) to, and get results back from.

Lesson 15: Introduction to Debugging

Debugging requires verifying dynamic values, tracking state changes across variable mutations, and isolating semantic errors within your processing blocks.

Pro-tip: Printing vs. Debugger

While `print()` tracing is great for beginners, professional IDEs like PyCharm have a built-in "Debugger" that lets you pause your code mid-execution to inspect the values of all variables at once without adding any extra print lines!